

Student API — A to Z Exam Guidance Handbook

Spring Boot · Postman · Git · Docker · Jenkins

Print this before the exam. Follow **Part A** in order (Start → End).

If something breaks, go to **Part C — Problems and Fixing** at the end.

Item	Value
Local folder	C:\Users\rasar\OneDrive\Desktop\SOC
GitHub	https://github.com/Razaara/student-api
Docker Hub	razzara/student-api
Port	8500 (API) · 3306 (MySQL) · 8080 (Jenkins) · 9000 (SonarQube)
Base URL	http://localhost:8500/api/students
DB name	students
DB user / pass	root / root
Java	21
Spring Boot	3.4.5
Jenkins job	student-api (branch */main)
Jenkins Maven tool	Maven
SonarQube server name	sonar
SonarQube project key	jenkins (must match your analysis token's project)
Jenkins Sonar credential ID	sonar

How this book is organized

Part	What it is
Part A	Project steps Start → End (do in this exact order)
Part B	Quick reference (annotations)
Part C	Problems and Fixing (Git, ports, Spring, Docker, Jenkins)
Part D	One-page final checklist

PART A — Project Steps (Start → End)

Do these steps in order. Do not skip ahead unless the examiner asks.

STEP 01 Create Spring Boot project (Desktop\SOC)
STEP 02 Project structure + .gitignore
STEP 03 pom.xml dependencies
STEP 04 application.properties (port 8500, DB root/root)
STEP 05 Main class (StudentApiApplication)
STEP 06 Entity → Repository → Service → Controller
STEP 07 Start MySQL + run app (docker compose / mvn)
STEP 08 Test with Postman (POST→GET→PUT→DELETE)
STEP 09 Git push to GitHub (Razaara/student-api)
STEP 10 Docker build / docker compose up (MySQL + API + SonarQube)
STEP 11 Push image to Docker Hub (razzara/student-api)
STEP 12 SonarQube token + Jenkins config
STEP 13 Jenkins Pipeline Build Now → verify API + Sonar

Aligned full flow (what you actually do on exam day):

```
Code CRUD
↓
MySQL up → mvn spring-boot:run → Postman test
↓
git add / commit / push (main)
↓
docker compose up -d --build (or docker build/run)
↓
docker tag + push razzara/student-api
↓
SonarQube running (:9000) → create/use project key "jenkins" → token
↓
Jenkins credential ID "sonar" + server name "sonar"
↓
Job student-api (branch */main, Jenkinsfile) → Build Now
↓
Stages: Checkout → Maven → Sonar → Quality Gate → Docker → Run → Verify
↓
Check: http://localhost:8500/api/students
      http://localhost:9000/dashboard?id=jenkins
```

If time is short: Steps 01–08 first → 09 Git → 10 Docker → 12–13 Jenkins/Sonar last.

STEP 01 — Create Spring Boot project

1. Open <https://start.spring.io> or IntelliJ → New → Spring Initializr
2. Project: **Maven** · Language: **Java** · Java: **21**
3. Group: `com.example` · Artifact: `studentapi` · Package: `com.example.studentapi`
4. Add dependencies:
 - Spring Web
 - Spring Data JPA
 - MySQL Driver
 - Lombok
 - Validation

5. Generate and extract into `C:\Users\rasar\OneDrive\Desktop\SOC`

*If Initializr only offers Boot 4.x, use Boot **3.4.x** if available, or keep parent version `3.4.5` as in this handbook.*

Already have the working repo?

```
cd C:\Users\rasar\OneDrive\Desktop
git clone https://github.com/Razaara/student-api.git SOC
cd SOC
```

STEP 02 — Project structure + .gitignore

```
SOC/
├─ pom.xml
├─ Dockerfile
├─ docker-compose.yml
├─ Jenkinsfile
├─ sonar-project.properties
├─ .gitignore
└─ src/main/
    ├─ java/com/example/studentapi/
    │   ├─ StudentApiApplication.java
    │   ├─ entity/Student.java
    │   ├─ repository/StudentRepository.java
    │   ├─ service/StudentService.java
    │   ├─ service/StudentServiceImpl.java
    │   └─ controller/StudentController.java
    └─ resources/application.properties
```

.gitignore

```
target/
.idea/
*.iml
*.class
.env
*.log
```

STEP 03 — pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.4.5</version>
<relativePath/>
</parent>

<groupId>com.example</groupId>
<artifactId>studentapi</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>studentapi</name>
<description>Student CRUD REST API for SOC practical exam</description>

<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
```

```
        <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
        </exclude>
    </excludes>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

STEP 04 — application.properties

File: src/main/resources/application.properties

```
spring.application.name=studentapi
spring.datasource.url=jdbc:mysql://localhost:3306/students?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
server.port=8500
```

Setting	Meaning
createDatabaseIfNotExist=true	Creates DB if missing
ddl-auto=update	Creates/updates tables from entity
server.port=8500	API port (not 8080)
password root	Matches Docker MySQL

STEP 05 — Main class

File: StudentApiApplication.java

```
package com.example.studentapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApiApplication.class, args);
    }
}
```

All other classes must stay under package `com.example.studentapi...` for component scanning.

STEP 06 — Entity → Repository → Service → Controller

Create in this order.

6.1 Entity — Student.java

```
package com.example.studentapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Entity
@Table(name = "student")
@AllArgsConstructor
@NoArgsConstructor
public class Student {

    @Id
    private String id;
    private String name;
    private String email;
    private Integer age;
}
```

Use `jakarta.persistence` (Boot 3). Send `id` in POST body (e.g. `AS2023000`).

6.2 Repository — StudentRepository.java

```
package com.example.studentapi.repository;

import com.example.studentapi.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, String> {
}
```

6.3 Service interface — StudentService.java

```
package com.example.studentapi.service;

import com.example.studentapi.entity.Student;
import java.util.List;
```

```

public interface StudentService {
    Student save(Student student);
    List<Student> getAll();
    Student getById(String id);
    Student update(String id, Student student);
    void delete(String id);
}

```

6.4 Service impl — StudentServiceImpl.java

```

package com.example.studentapi.service;

import com.example.studentapi.entity.Student;
import com.example.studentapi.repository.StudentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class StudentServiceImpl implements StudentService {

    private final StudentRepository studentRepository;

    @Override
    public Student save(Student student) {
        return studentRepository.save(student);
    }

    @Override
    public List<Student> getAll() {
        return studentRepository.findAll();
    }

    @Override
    public Student getById(String id) {
        return studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student not found with id: " +
id));
    }

    @Override
    public Student update(String id, Student student) {
        Student existing = getById(id);
        existing.setName(student.getName());
        existing.setEmail(student.getEmail());
        existing.setAge(student.getAge());
        return studentRepository.save(existing);
    }
}

```

```

@Override
public void delete(String id) {
    studentRepository.deleteById(id);
}
}

```

6.5 Controller — StudentController.java

```

package com.example.studentapi.controller;

import com.example.studentapi.entity.Student;
import com.example.studentapi.service.StudentService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping
    public ResponseEntity<Student> create(@RequestBody Student student) {
        Student saved = studentService.save(student);
        return new ResponseEntity<>(saved, HttpStatus.CREATED);
    }

    @GetMapping
    public ResponseEntity<List<Student>> getAll() {
        return ResponseEntity.ok(studentService.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable String id) {
        return ResponseEntity.ok(studentService.getById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> update(@PathVariable String id, @RequestBody Student
student) {
        return ResponseEntity.ok(studentService.update(id, student));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        studentService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```



```
}  
}
```

Endpoints (remember these)

Method	Path	Success
POST	/api/students	201
GET	/api/students	200
GET	/api/students/{id}	200
PUT	/api/students/{id}	200
DELETE	/api/students/{id}	204

STEP 07 — Start MySQL + run the app

7.1 Start MySQL (Docker)

```
cd C:\Users\rasar\OneDrive\Desktop\SOC  
docker compose up -d mysql  
docker ps
```

Wait until `student-mysql` is **healthy**.

7.2 Run Spring Boot

```
mvn clean package -DskipTests  
mvn spring-boot:run
```

Console must show:

```
Tomcat started on port(s): 8500  
Started StudentApiApplication
```

*Port busy? Jump to **Part C** → **Ports**.*

STEP 08 — Test with Postman

Base URL: `http://localhost:8500/api/students`

Body: **raw** → **JSON**

8.1 POST — Create (201)

POST `http://localhost:8500/api/students`

```
{  
  "id": "AS2023000",
```

```
"name": "John Doe",  
"email": "john@example.com",  
"age": 21  
}
```

8.2 GET — All (200)

```
GET http://localhost:8500/api/students
```

8.3 GET — By id (200)

```
GET http://localhost:8500/api/students/AS2023000
```

8.4 PUT — Update (200)

```
PUT http://localhost:8500/api/students/AS2023000
```

```
{  
  "name": "John Updated",  
  "email": "john.updated@example.com",  
  "age": 22  
}
```

8.5 DELETE (204)

```
DELETE http://localhost:8500/api/students/AS2023000
```

Test order

```
POST (201) → GET all (200) → GET by id (200) → PUT (200) → DELETE (204)
```

STEP 09 — Git commit + push to GitHub

There are **two ways** to get code onto GitHub. Use the one that matches your exam situation.

Option A — Create project on Desktop first (NO clone), then push

Use this when you create a folder on Desktop, build the microservice there, then upload to GitHub.

A1. Go to your project folder

```
cd C:\Users\rasar\OneDrive\Desktop\YOUR_FOLDER_NAME
```

Example (this project):

```
cd C:\Users\rasar\OneDrive\Desktop\SOC
```

After clone/create, `SOC` **is** the student-api project root. There is no extra inner `student-api` folder unless you created one yourself.

A2. Initialize Git (first time only)

```
git init
git branch -M main
```

A3. Make sure `.gitignore` exists

```
target/
.idea/
*.class
.env
*.log
```

A4. Create empty repo on GitHub

1. Open <https://github.com> → login (**Razaara**)
2. **New repository**
3. Name: e.g. `student-api`
4. Keep it **empty** (do not add README if code already exists locally)
5. Create repository
6. Copy URL, example:

```
https://github.com/Razaara/student-api.git
```

A5. Connect local folder to GitHub

```
git remote add origin https://github.com/Razaara/student-api.git
```

If remote already exists / wrong URL:

```
git remote set-url origin https://github.com/Razaara/student-api.git
```

Check:

```
git remote -v
```

A6. First push

```
git add .
git commit -m "Initial commit: Student microservice"
git push -u origin main
```

Option B — Clone existing GitHub repo first, then work

Use this when the repo already exists on GitHub and you want a local copy.

```
cd C:\Users\rasar\OneDrive\Desktop
git clone https://github.com/Razaara/student-api.git SOC
```

```
cd SOC
```

Then edit files inside `SOC` and push updates (see “Everyday push” below).

Everyday push (after you add/edit files)

Works for both Option A and Option B:

```
cd C:\Users\rasar\OneDrive\Desktop\SOC
git status
git add .
git commit -m "Update project"
git push origin main
```

Shared lab PC (before push)

1. Credential Manager → Windows Credentials → remove `github.com`
2. Then:

```
git config user.name "Your Name"
git config user.email "your-email@example.com"
git remote -v
git push -u origin main
```

Login with **your** GitHub account when prompted.
(`user.name` / `user.email` do **not** authenticate you.)

This project repo: <https://github.com/Razaara/student-api>

*Git errors? Jump to **Part C** → **Git**.*

STEP 10 — Docker

10.1 Dockerfile

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8500
ENTRYPOINT ["java", "-jar", "app.jar"]
```

10.2 Build & run one container (MySQL already running)

```

docker build -t student-api .
docker run -d -p 8500:8500 --name student-api-container ^
  -e SPRING_DATASOURCE_URL=jdbc:mysql://host.docker.internal:3306/students?
createDatabaseIfNotExist=true ^
  -e SPRING_DATASOURCE_USERNAME=root ^
  -e SPRING_DATASOURCE_PASSWORD=root ^
  student-api

docker logs -f student-api-container

```

10.3 docker-compose.yml (MySQL + API + SonarQube)

```

services:
  mysql:
    image: mysql:8.0
    container_name: student-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: students
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s

  springboot:
    build: .
    container_name: student-api
    restart: on-failure
    ports:
      - "8500:8500"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/students?createDatabaseIfNotExist=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
    depends_on:
      mysql:
        condition: service_healthy

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    restart: unless-stopped
    ports:

```

```

    - "9000:9000"
environment:
  SONAR_ES_BOOTSTRAP_CHECKS_DISABLE: "true"
volumes:
  - sonarqube_data:/opt/sonarqube/data
  - sonarqube_extensions:/opt/sonarqube/extensions
  - sonarqube_logs:/opt/sonarqube/logs

volumes:
  mysql_data:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:

```

Compose commands

```

cd C:\Users\rasar\OneDrive\Desktop\SOC

# If an old standalone Sonar container exists, remove it first:
docker stop sonarqube 2>NUL
docker rm sonarqube 2>NUL

# Start MySQL + API + SonarQube together
docker compose up -d --build

docker compose ps
docker compose logs -f springboot
docker compose logs -f sonarqube
docker compose down

```

Start only some services

```

# MySQL only
docker compose up -d mysql

# MySQL + Sonar (no API container)
docker compose up -d mysql sonarqube

# Sonar only
docker compose up -d sonarqube

```

Service	URL / Port
API	http://localhost:8500/api/students
SonarQube	http://localhost:9000
MySQL	localhost:3306

Compose DB host = service name `mysql` . Single `docker run` DB host = `host.docker.internal` .
SonarQube needs **1–2 minutes** after start before <http://localhost:9000> loads.

STEP 11 — Push image to Docker Hub

Docker user: `razzara`

```
docker login
docker tag student-api:latest razzara/student-api:latest
docker push razzara/student-api:latest
```

```
docker pull razzara/student-api:latest
```

STEP 12 — SonarQube token + Jenkins config

12.1 Start services

```
cd C:\Users\rasar\OneDrive\Desktop\SOC
docker compose up -d mysql sonarqube
# or everything: docker compose up -d --build
```

- SonarQube: <http://localhost:9000>
- Jenkins: <http://localhost:8080>
- MySQL: port **3306**

12.2 Get SonarQube token (atomic)

1. Login SonarQube
2. Open/create project with key `jenkins` (must match your analysis token)
3. **My Account** → **Security** → **Generate Token** → copy token

12.3 Put token into Jenkins (atomic)

1. Jenkins → **Manage Jenkins** → **Credentials** → **System** → **Global credentials**
2. Add/Update:
 - Kind: **Secret text**
 - Secret: *(paste token)*
 - ID: `sonar`
3. **Manage Jenkins** → **System** → **SonarQube servers**
 - Name: `sonar`
 - URL: `http://localhost:9000`
 - Token credential: `sonar`
4. Save
5. (Recommended) SonarQube → **Administration** → **Webhooks**
 - URL: `http://localhost:8080/sonarqube-webhook/`

12.4 Create / check Jenkins job (atomic)

1. **New Item** → name `student-api` → **Pipeline**

2. Definition: **Pipeline script from SCM**
3. Git URL: `https://github.com/Razaara/student-api.git`
4. Branch: `*/main` (not master)
5. Script Path: `Jenkinsfile`
6. Save

12.5 Tools must match

- Maven tool name: `Maven`
- Sonar server name: `sonar`
- Sonar project key in Jenkinsfile: `jenkins`

STEP 13 — Build pipeline + verify

13.1 Before Build Now

```
docker compose up -d mysql
docker stop student-api student-api-container 2>NUL
docker rm student-api student-api-container 2>NUL
docker start sonarqube
```

13.2 Build Now

Jenkins → job **student-api** → **Build Now**

13.3 Pipeline stages (aligned)

- 1 Checkout
- 2 Build with Maven
- 3 SonarQube Analysis ← project key "jenkins"
- 4 Quality Gate ← webhook recommended; won't abort whole job
- 5 Build Docker Image
- 6 Stop Old Container
- 7 Remove Old Container
- 8 Run New Container ← port 8500 + host.docker.internal MySQL
- 9 Verify ← GET /api/students

13.4 After SUCCESS

Check	URL
API	http://localhost:8500/api/students
Sonar dashboard	http://localhost:9000/dashboard?id=jenkins
Jenkins job	http://localhost:8080/job/student-api/

13.5 Current working Jenkinsfile (summary env)


```
environment {
    IMAGE_NAME = 'student-api'
    CONTAINER_NAME = 'student-api-container'
    SONARQUBE_SERVER = 'sonar'
    SONAR_PROJECT_KEY = 'jenkins'
    SONAR_PROJECT_NAME = 'jenkins'
    DB_URL = 'jdbc:mysql://host.docker.internal:3306/students?createDatabaseIfNotExist=true'
    DB_USER = 'root'
    DB_PASS = 'root'
}
```

Your Sonar analysis token must belong to project key `jenkins` .

If you create a new project/token with another key, change `SONAR_PROJECT_KEY` to match.

PART B — Quick Reference (Annotations)

Annotation	Use
@SpringBootApplication	Main class
@RestController	REST controller
@RequestMapping	Base path /api/students
@PostMapping / @GetMapping / @PutMapping / @DeleteMapping	HTTP methods
@PathVariable	{id} from URL
@RequestBody	JSON → Java object
@Service	Service bean
@Entity / @Id / @Table	JPA entity
@Data	Lombok getters/setters
@RequiredArgsConstructor	Constructor injection

PART C — Problems and Fixing (FINAL)

Use this part only when something fails. Find your error → apply the fix → return to Part A.

C1. Quick problem index

Area	Go to
Spring Boot / MySQL / Postman	C2
Git / GitHub	C3

Port already in use (8500 / 3306 / 8080 / 9000)	C4
Docker / Docker Hub	C5
Jenkins / SonarQube	C6

C2. Spring Boot / MySQL / Postman problems

Problem	Fix
Communications link failure	<code>docker compose up -d mysql</code> — wait until healthy
Access denied for user 'root'	Set password to root in <code>application.properties</code>
Unknown database	Keep <code>createDatabaseIfNotExist=true</code>
Table doesn't exist	Keep <code>spring.jpa.hibernate.ddl-auto=update</code>
Whitelabel 404	Check URL = <code>/api/students</code> and controller mapping
400 Bad Request on POST	Valid JSON + include "id" field; use Postman raw JSON
500 on GET missing id	Expected with sample code (<code>RuntimeException</code>)
<code>javax.persistence</code> errors	Change imports to <code>jakarta.persistence</code>
Lombok getters not found	Install Lombok plugin + enable annotation processing
App starts but Postman ECONNREFUSED	Confirm console: Tomcat started on port(s): 8500

C3. Common Git issues

Problem	Cause	Fix
fatal: not a git repository	Wrong folder / no init	<code>cd project</code> → <code>git init</code>
remote origin already exists	Origin already added	<code>git remote set-url origin https://github.com/Razaara/student-api.git</code>
Updates rejected (non-fast-forward)	Remote ahead of local	<code>git pull origin main</code> then <code>git push</code>
Authentication failed	GitHub blocks password	Use PAT / Git Credential Manager login
Wrong GitHub account (lab PC)	Old Windows credentials	Credential Manager → remove <code>github.com</code> → push → login again
Permission denied (publickey)	SSH not set up	Use HTTPS remote URL
Everything up-to-date but GitHub empty	Forgot commit	<code>git add .</code> → <code>git commit -m "msg"</code> → <code>git push</code>

Committed target/	Missing .gitignore	Add target/ → git rm -r --cached target/ → commit → push
Merge conflict <<<<<<	Same lines edited twice	Edit file → remove markers → git add . → git commit
Unrelated histories	New local + existing remote	git pull origin main --allow-unrelated-histories
Please tell me who you are	Identity missing	git config user.name "..." and user.email "..."
Detached HEAD	Checked out a commit	git checkout main

Git recovery commands

```
git status
git remote -v
git log --oneline -5

git remote set-url origin https://github.com/Razaara/student-api.git
git pull origin main
git push origin main

git rm -r --cached target/
git add .
git commit -m "Stop tracking target folder"
git push
```

Shared PC Git checklist

1. Credential Manager → remove github.com
2. git config user.name "Your Name"
3. git config user.email "your-email@example.com"
4. git remote -v (must be YOUR repo)
5. git add . → commit → push
6. Login with YOUR GitHub account

`user.name` / `user.email` do **not** log you into GitHub. Login/PAT does.

C4. Ports already in use — how to fix

Port	Used by
8500	Spring Boot API
3306	MySQL
8080	Jenkins
9000	SonarQube

Symptoms

- Port 8500 was already in use
- Bind for 0.0.0.0:8500 failed: port is already allocated

Find what uses the port

```
netstat -ano | findstr :8500
```

Last column = **PID**.

Fix A — Stop Docker containers (most common)

```
docker ps
docker stop student-api student-api-container student-mysql 2>NUL
docker rm student-api student-api-container student-mysql 2>NUL
cd C:\Users\rasar\OneDrive\Desktop\SOC
docker compose down
```

Then restart what you need:

```
docker compose up -d mysql
```

Fix B — Kill Java / local process

```
netstat -ano | findstr :8500
taskkill /PID <PID> /F
```

Or:

```
Get-NetTCPConnection -LocalPort 8500 -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction SilentlyContinue }
```

Fast fix — port 8500

```
docker stop student-api student-api-container 2>NUL
docker rm student-api student-api-container 2>NUL
netstat -ano | findstr :8500
taskkill /PID <PID> /F
mvn spring-boot:run
```

Fast fix — port 3306

```
docker stop student-mysql 2>NUL
docker rm student-mysql 2>NUL
docker compose up -d mysql
```

Fast fix — port 8080 (Jenkins)

```
netstat -ano | findstr :8080
Get-Service Jenkins
Start-Service Jenkins
```

Last resort — change app port

```
server.port=8501
```

Then Postman: `http://localhost:8501/api/students`

Prefer freeing the port so handbook URLs stay `8500` .

C5. Docker / Docker Hub problems

Problem	Fix
Cannot connect to Docker daemon	Start Docker Desktop
Port already allocated	See C4
Communications link failure in container	Use <code>host.docker.internal</code> (docker run) or <code>mysql</code> (compose)
Container exits immediately	<code>docker logs <name></code> — read startup error
Push denied	<code>docker login</code> then <code>tag razzara/student-api</code>
Old code still running	<code>docker build --no-cache -t student-api .</code>
No space left	<code>docker system prune -a</code>

C6. Jenkins / SonarQube problems

Problem	Fix
<code>mvn</code> : command not found	Tools name must be exactly Maven
<code>docker</code> : command not found	Install/start Docker; restart Jenkins
Checkout auth failed	Public repo URL correct; or add GitHub credentials
Port 8500 conflict on Run stage	Stop old containers (see C4) before Build Now
Container can't reach MySQL	Keep <code>host.docker.internal</code> in Jenkinsfile env
Pipeline red / Verify failed	Open Console Output + <code>docker logs student-api-container</code>
<code>withSonarQubeEnv</code> / server not found	Jenkins System → SonarQube servers Name must be sonar
Sonar analysis auth failed	Add Sonar token credential + select it on SonarQube server config

Cannot connect to SonarQube	Start SonarQube; open http://localhost:9000
Quality Gate stuck waiting	Add Sonar webhook → http://localhost:8080/sonarqube-webhook/
Quality Gate failed but you still need deploy	Keep abortPipeline: false (as in this Jenkinsfile)

C7. HTTP status meanings

Code	Meaning
200	OK (GET / PUT)
201	Created (POST)
204	No Content (DELETE)
400	Bad request / bad JSON
404	Wrong URL / not mapped
500	Server exception

PART D — One-Page Final Checklist

Start → End command strip

```
# STEP 07
docker compose up -d mysql
mvn spring-boot:run

# STEP 08 – Postman: POST → GET → GET id → PUT → DELETE

# STEP 09 – if created locally first (no clone):
# git init → git branch -M main → create empty GitHub repo →
# git remote add origin <your-repo-url> → git add . → commit → push -u origin main
#
# Everyday push after adding files:
git add .
git commit -m "Student CRUD API"
git push origin main

# STEP 10-11
docker build -t student-api .
docker compose up -d --build
docker tag student-api razzara/student-api
docker push razzara/student-api

# STEP 12
# Start SonarQube (http://localhost:9000)
```

```
# http://localhost:8080 → student-api → Build Now
# Check Sonar project: student-api
```

Remember

- Follow **Part A steps in order**
- Port **8500** everywhere · SonarQube **9000** · Jenkins **8080**
- Include **id** in POST body
- `jakarta.persistence` not `javax`
- Compose DB host = `mysql`
- Jenkins DB host = `host.docker.internal`
- SonarQube server name in Jenkins = `sonar`
- GitHub: **Razaara** · Docker: **razzara**
- Broken? → **Part C Problems and Fixing**

Viva quick answers

- `@Component` vs `@Service` vs `@Repository` — same mechanism, different layer meaning
- Dependency Injection — Spring injects beans (constructor / `@RequiredArgsConstructor`)
- `JpaRepository` vs `CrudRepository` — Jpa adds paging/sorting helpers
- Image vs Container — image = template; container = running instance
- Declarative pipeline — `pipeline { stages { ... } }` in Jenkinsfile

End of handbook.

Project: `C:\Users\rasar\OneDrive\Desktop\SOC`

Repo: <https://github.com/Razaara/student-api>